

BLOC C • SÉANCES 5 À 6

Prompt, tokens et coûts

Écrire une consigne, et savoir ce qu'elle coûte



Les séances de ce bloc

1

Séance 5

Le prompt comme interface de programmation

2

Séance 6

Ce que ça coûte vraiment, en cloud et en local

SÉANCE 5 • PROMPT ET COÛTS

Le prompt comme interface de programmation

Écrire une consigne qui tienne, et savoir qu'elle tient

Au programme aujourd'hui

1

Cadrer

Le prompt n'est pas une conversation, c'est un contrat

2

Structurer

Les quatre morceaux d'une consigne qui tient

3

Contraindre

Obtenir du JSON exploitable, à tous les coups

4

Mesurer

Un prompt s'évalue sur des cas, pas à l'intuition

Vous n'écrivez pas un message, vous appelez une fonction

- Le prompt est le seul paramètre d'entrée de la fonction la plus imprévisible de votre code.
- Il se versionne, se teste et se relit en revue de code.
- Un prompt jeté dans une chaîne au milieu d'un contrôleur est une dette technique.



Traitez-le comme du code

Un fichier dédié, un nom, une version, un jeu de cas. Exactement ce que vous feriez pour une expression régulière critique.

Quatre morceaux, dans cet ordre

R

Rôle

Qui parle et pour quoi faire. Une phrase suffit, placée dans le message système.

C

Contexte

Les données du cas précis. Séparées de la consigne, jamais mélangées.

F

Format

La forme exacte de la sortie attendue. Montrez-la, ne la décrivez pas.

E

Exemples

Un à trois cas résolus. Le levier le plus rentable de tous.

Un prompt structuré, en Python

```
SYSTEME = """Tu classes des tickets de support client.  
Reponds uniquement par un objet JSON de cette forme :  
{"categorie": "paiement|livraison|compte", "urgence": 1}  
N'ajoute aucun texte avant ou apres le JSON."""  
  
reponse = httpx.post(f"{BASE}/v1/chat/completions", json={  
    "model": MODELE,  
    "temperature": 0,           # classification : pas de hasard  
    "messages": [  
        {"role": "system", "content": SYSTEME},  
        {"role": "user", "content": ticket},  
    ],  
}, timeout=60).json()
```

Le format est montré, pas décrit : c'est ce qui fait la différence entre du JSON valide et du texte qui y ressemble.

Le même appel, deux syntaxes

Même URL, même corps de requête, même réponse. Seule la syntaxe diffère.

PYTHON

```
r = httpx.post(URL, json={
    "model": MODELE,
    "temperature": 0,
    "messages": messages,
})

d = r.json()
texte = d["choices"][0]\
    ["message"]["content"]
```

JAVASCRIPT

```
const r = await fetch(URL, {
  method: "POST",
  headers: { "Content-Type":
    "application/json" },
  body: JSON.stringify({
    model: MODELE,
    temperature: 0, messages,
  }),
});
const d = await r.json();
const texte =
  d.choices[0].message.content;
```

Le contrat est à moi, le langage est à vous

- Les démonstrations et les corrigés sont en Python : c'est ma langue.
- Vos rendus sont dans la langue que vous voulez : c'est votre métier.
- Ce qui est évalué est le contrat HTTP, identique pour tout le monde.



Une suite de tests, toutes les langues

Vos serveurs seront testés par des requêtes HTTP. Un test qui passe passe en TypeScript comme en Python.

Quel levier pour quel problème

Symptôme	Levier	Coût
Sortie mal formée	montrer le format	nul
Réponses à côté	un à trois exemples	quelques tokens
Erreurs de logique	demander les étapes	plus de tokens
Trop bavard	imposer une longueur	nul
Invente des faits	fournir les sources	beaucoup

On monte dans le tableau avant de descendre : les leviers gratuits d'abord.

Ce qui casse un prompt sans prévenir

Quatre erreurs que vous ferez toutes, au moins une fois.

- **L'ambiguïté** : « sois concis » ne veut rien dire ; « trois phrases maximum » se vérifie
- **La contradiction** : une consigne système et un exemple qui se contredisent : l'exemple gagne
- **Le mélange** : des données utilisateur collées dans la consigne deviennent des instructions
- **La dépendance au modèle** : un prompt réglé sur un modèle se dégrade sur un autre : à retester

Faire passer un prompt de 0 % à 90 % de réussite

- 1 Récupérer le jeu de 20 tickets et le classifieur de départ.
- 2 Mesurer le taux de réussite du prompt fourni, tel quel.
- 3 Améliorer par étapes : format, puis exemples, puis rôle.
- 4 Remesurer après chaque étape et noter le gain.
- 5 Garder la trace des versions : elle est demandée au CC2.

Objectif

Comprendre qu'un prompt se travaille par mesure, pas par intuition. Le petit modèle local rend les progrès très visibles.

Le prompt comme interface

- ✓ Un prompt est du code : versionné, testé, relu.
- ✓ Quatre morceaux : rôle, contexte, format montré, exemples.
- ✓ On contraint le format en le montrant, et on valide la sortie côté serveur.
- ✓ Un prompt s'évalue sur un jeu de cas, jamais sur une impression.

SÉANCE 6 • PROMPT ET COÛTS

Ce que ça coûte vraiment, en cloud et en local

Chiffrer une fonctionnalité avant de l'écrire

Au programme aujourd'hui

1

Compter

La facturation au token, entrée et sortie

2

Comparer

À partir de quel volume le local devient rentable

3

Attendre

La latence perçue, et ce que le streaming y change

4

Chiffrer

Le budget d'une fonctionnalité pour 10 000 utilisateurs

Comment un fournisseur vous facture

Trois règles suffisent à estimer une facture à 20 % près.

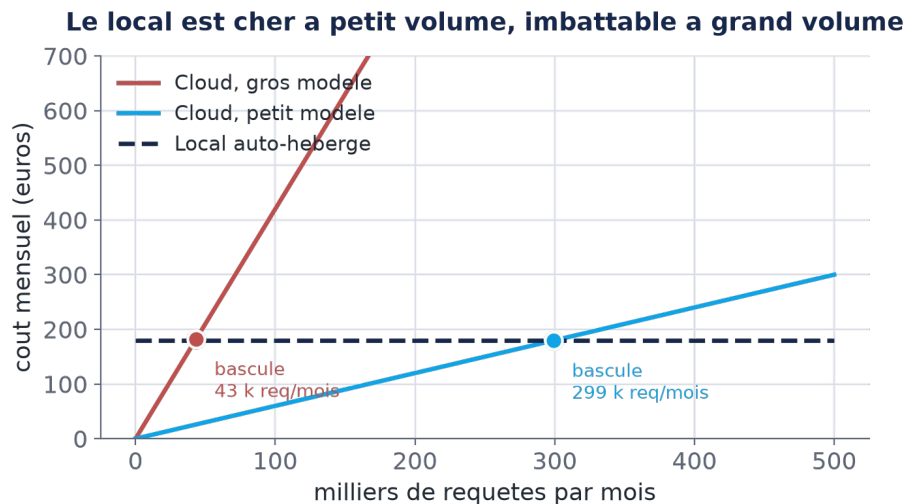
- **Tout est compté** : votre prompt, l'historique renvoyé, les documents joints, et la réponse
- **La sortie coûte plus cher** : souvent trois à cinq fois le prix de l'entrée, pour le même nombre de tokens
- **Le contexte se repaie à chaque tour** : dans un chat, l'historique est refacturé à chaque message
- **Le cache change la donne** : un préfixe stable et réutilisé peut être facturé une fraction du prix

Ce qu'il faut avoir en tête, pas par coeur

Classe de modèle	Entrée / M tokens	Sortie / M tokens
Très petit, cloud	0,05 à 0,20 €	0,20 à 0,60 €
Moyen, cloud	0,50 à 3 €	1,50 à 15 €
Haut de gamme, cloud	3 à 15 €	15 à 75 €
Local, sur votre poste	0 €	0 €

Un facteur cent sépare les extrêmes. Le choix du modèle pèse plus lourd que toutes vos optimisations de prompt.

Le local n'est pas gratuit, il est à coût fixe



Le cloud coûte zéro à volume zéro, puis croît sans limite.

Le local coûte le même prix tous les mois, que vous l'utilisiez ou non.

En local, le coût change de nature

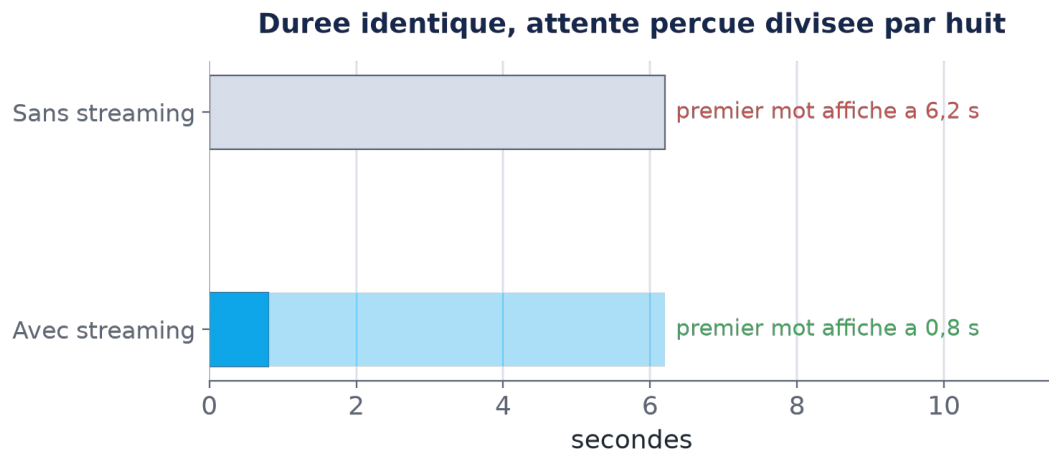
- Vous ne payez plus au token : vous payez une machine et son administration.
- La contrainte devient le débit : combien de requêtes simultanées la machine tient.
- Un modèle local saturé ne coûte pas plus cher, il devient lent.



Mesurez votre débit

Le chiffre en tokens par seconde relevé en séance 4 est votre capacité. Sortez-le maintenant.

Le streaming ne va pas plus vite, il se fait pardonner



La durée totale ne change pas. Ce qui change, c'est le moment où l'utilisateur voit quelque chose arriver.

Compter avant d'envoyer, mesurer après

```
# le serveur renvoie le compte exact : servez-vous en  
d = httpx.post(URL, json=corps, timeout=120).json()  
u = d["usage"]
```

```
entree, sortie = u["prompt_tokens"], u["completion_tokens"]  
cout = (entree * PRIX_E + sortie * PRIX_S) / 1_000_000
```

```
print(f"{entree} + {sortie} tokens, soit {cout:.5f} EUR")  
70 + 214 tokens, soit 0.00034 EUR
```

Journalisez ces deux nombres à chaque appel dès le premier jour : sans eux, aucune estimation n'est possible.

Réduire la facture, du plus rentable au moins

Levier	Gain typique	Ce que ça coûte
Prendre plus petit	x10 à x100	un peu de qualité
Plafonner max_tokens	x2 sur la sortie	réponses tronquées
Tailler l'historique	x2 à x5	le modèle oublie
Cacher le préfixe	x2 sur l'entrée	rien, si stable
Raccourcir le prompt	quelques %	du temps d'ingénieur

Le budget d'une vraie fonctionnalité

- 1 Fonctionnalité : résumer un ticket, 3 requêtes par utilisateur et par mois.
- 2 Hypothèse : 10 000 utilisateurs actifs, 600 tokens en entrée, 150 en sortie.
- 3 Chiffrer le coût mensuel avec un modèle cloud haut de gamme.
- 4 Refaire le calcul avec un petit modèle cloud, puis en local auto-hébergé.
- 5 Conclure : à quel volume basculez-vous, et qu'est-ce qui vous en empêche ?

Objectif

Savoir défendre un choix d'architecture avec des chiffres. Ce calcul est la deuxième partie du CC1.

Ce que ça coûte

- ✓ Tout est compté en tokens, et la sortie coûte plus cher que l'entrée.
- ✓ Le cloud est proportionnel au volume, le local est à coût fixe.
- ✓ Le premier levier est le choix du modèle, très loin devant les autres.
- ✓ Le streaming ne réduit pas la durée, il réduit l'attente perçue.